
REUSABLE ENGINEERING HANDBOOK

Angular Testing and Quality Automation

A project-independent, step-by-step implementation guide for Vitest, Angular TestBed, code coverage, Allure reports, quality gates, GitHub Actions and optional IIS deployment.

Use this guide for: new Angular applications or existing applications migrating to Vitest

Audience: beginners, developers, reviewers and CI/CD maintainers

Edition: 1.0 | **Published:** September 2026

Scope: concepts, implementation templates, verification and troubleshooting

Contents

1. What this handbook builds

2. Conceptual model

3. Compatibility assessment

4. Install the toolchain

5. Configure Angular and Vitest

6. Define npm commands

7. Write maintainable unit tests

8. Components and DOM tests

9. Services and HTTP mocks

10. Pipes, directives and utilities

11. Coverage policy

12. Allure integration

13. Quality gates

14. Linting and formatting

15. GitHub Actions CI

16. Artifacts and CI diagnosis

17. Optional IIS deployment

18. Verification checklist

19. Troubleshooting

20. Rollout and maintenance

21. Copy-paste support scripts

22. Glossary and references

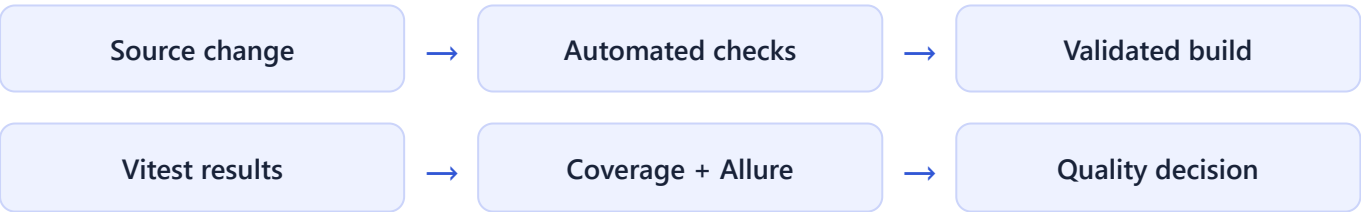
1. What this handbook builds

This handbook describes a reusable testing platform rather than one application's tests. Replace placeholders such as <project-name>, <app-name>, <node-version> and <coverage-target> with values agreed by your team.

Target outcome: Every developer and CI runner can install identical dependencies, run deterministic Angular unit tests, enforce coverage and test-quality rules, inspect Allure reports, build only after required checks pass and preserve diagnostic artifacts.

Layer	Question answered	Typical tool
Unit tests	Does this behavior produce the expected result?	Vitest + Angular TestBed
Coverage	Which source paths were executed?	V8 coverage
Results	Which cases passed, failed or took time?	Allure Report
Static quality	Does code follow safety and style rules?	ESLint + Prettier
CI	Do the same checks pass on every change?	GitHub Actions
Delivery	Can a validated artifact be deployed?	Angular build + chosen host

2. Conceptual model



Vitest decides pass or fail for assertions. Coverage measures execution. Allure visualizes results. ESLint and Prettier check maintainability. CI runs the same commands outside a developer's machine. These responsibilities overlap intentionally but are not interchangeable.

Continuous Integration

Automatically validates commits and pull requests on a clean runner. The output is a pass/fail status plus diagnostic artifacts.

Continuous Delivery/Deployment

Packages or publishes a validated application. Deployment requires an explicit target and credentials; CI alone does not create a public URL.

3. Compatibility assessment

3.1 Inspect the project first

```
node --version
npm --version
npx ng version
npm ls vitest jsdom @angular/build
```

New Angular CLI projects use Vitest and jsdom by default. Older applications may still contain Karma/Jasmine configuration. Look for:

Indicator	Meaning	Action
@angular/build:unit-test	Modern Angular test builder	Continue with this guide.
karma.conf.js or Karma builder	Existing Karma setup	Follow Angular's official Karma-to-Vitest migration before removing dependencies.
jasmine.createSpy	Jasmine-specific test API	Replace with <code>vi.fn()</code> or <code>vi.spyOn()</code> .
Custom browser APIs	jsdom may not implement required behavior	Mock the API or configure Vitest browser mode.

Version rule: Keep `vitest` and `@vitest/coverage-v8` on the same major version. Use package versions compatible with the Angular version in the target repository; do not copy version numbers blindly.

4. Install the toolchain

4.1 Create a new Angular project

```
npx @angular/cli@latest new <app-name> --style=scss
cd <app-name>
ng test
```

Confirm the generated test passes before adding reporting or quality tooling. This creates a known-good baseline.

4.2 Existing modern Angular project

```
npm install
ng test
```

4.3 Install coverage and reporting

```
npm install --save-dev @vitest/coverage-v8
npm install --save-dev allure allure-vitest allure-js-commons
```

4.4 Install quality tools when absent

```
ng add angular-eslint
npm install --save-dev prettier @vitest/eslint-plugin globals
```

Commit `package.json` and `package-lock.json`. CI should use `npm ci`, which installs the exact dependency graph recorded in the lockfile.

5. Configure Angular and Vitest

5.1 Generate a supported base config

```
ng generate config vitest
```

Angular creates a Vitest base file. Rename only if desired, then reference it from the target project's test options in `angular.json`.

5.2 Angular test target template

```
"test": {
  "builder": "@angular/build:unit-test",
  "options": {
    "progress": false,
    "runnerConfig": "vitest.config.ts",
    "isolate": true,
    "coverageInclude": ["src/app/**/*.ts", "src/app/**/*.html"],
    "coverageExclude": [
      "**/*.spec.ts",
      "**/*.model.ts",
      "**/app.config*.ts",
      "**/app.routes*.ts"
    ],
    "coverageReporters": ["text", "text-summary", "html", "lcov", "json-summary"],
    "coverageThresholds": {
      "perFile": true,
      "statements": <coverage-target>,
      "branches": <coverage-target>,
      "functions": <coverage-target>,
      "lines": <coverage-target>
    }
  }
}
```

Start with a target your existing codebase can honestly meet, then ratchet upward. A new small library may use 100%; a legacy application may adopt a lower baseline plus a “no decrease” policy.

5.3 Reusable Vitest configuration

```
import { defineConfig } from 'vitest/config';

export default defineConfig({
  test: {
    allowOnly: false,
    passWithNoTests: false,
    clearMocks: true,
    restoreMocks: true,
    retry: 0,
    reporters: [
      ['tree', { summary: false }],
      ['json', { outputFile: 'test-results/vitest.json' }],
      ['allure-vitest/reporter', {
        resultsDir: 'allure-results',
        environmentInfo: {
          node_version: process.version,
          platform: process.platform,
          execution: process.env['CI'] ? 'CI' : 'local'
        }
      }],
    ],
    coverage: {
      provider: 'v8',
      reportOnFailure: true
    }
  }
});
```

Option	Protection
allowOnly: false	Rejects accidentally committed focused tests.
passWithNoTests: false	Empty discovery cannot report success.
clearMocks / restoreMocks	Reduces cross-test state leakage.
retry: 0	Exposes unstable tests instead of hiding them.
reportOnFailure	Creates coverage evidence even when a test fails.

6. Define npm commands

Add scripts to package.json. Preserve existing project-specific scripts.

```
"scripts": {
  "test": "ng test --no-watch",
  "test:watch": "ng test --watch",
  "test:coverage": "ng test --coverage --no-watch",
  "test:ci": "npm run test:coverage && node scripts/check-test-results.mjs",
  "allure:generate": "allure generate allure-results --output allure-report --report-name \\\"Unit tests\\\"\"",
  "allure:open": "allure open allure-report",
  "allure:clean": "node scripts/clean-reports.mjs all",
  "lint": "eslint . --max-warnings=0",
  "lint:fix": "eslint . --fix",
  "format:check": "prettier --check .",
  "format": "prettier --write .",
  "quality-check": "npm run lint && npm run format:check && npm run test:ci && npm run allure:generate && npm run build"
}
```

Add pretest, pretest:coverage and preallure:generate cleanup hooks when repeatable report output is required. Do not clean results while a watch session is writing into them.

7. Write maintainable unit tests

7.1 Arrange–Act–Assert

```
it('calculates the discounted total', () => {  
  // Arrange  
  const price = 100;  
  const discount = 20;  
  
  // Act  
  const result = calculateDiscount(price, discount);  
  
  // Assert  
  expect(result).toBe(80);  
});
```

7.2 Naming convention

Use `*.spec.ts` beside the source file. Test names should describe observable behavior, for example *should reject an empty email*. Avoid vague names such as *test 1*.

7.3 Scenario selection

Scenario	Example	Purpose
Positive	Valid form submits	Confirms expected behavior.
Negative	Empty required field is rejected	Confirms validation and safe failure.
Boundary	Zero, maximum length, first/last item	Checks behavior at limits.
State transition	Edit → cancel → add mode	Confirms related state resets correctly.
Dependency failure	HTTP 500 response	Confirms error handling.

7.4 Independence rules

- Create fresh state in `beforeEach`.
- Do not depend on test execution order.
- Do not call real production APIs.
- Control clocks, randomness and asynchronous work.
- Restore fake timers and spies.
- Assert outcomes, not private implementation details.

8. Components and DOM tests

8.1 Standalone component setup

```
import { ComponentFixture, TestBed } from '@angular/core/testing';
import { ProfileComponent } from '../profile.component';

describe('ProfileComponent', () => {
  let fixture: ComponentFixture<ProfileComponent>;
  let component: ProfileComponent;

  beforeEach(async () => {
    await TestBed.configureTestingModule({
      imports: [ProfileComponent]
    }).compileComponents();

    fixture = TestBed.createComponent(ProfileComponent);
    component = fixture.componentInstance;
    fixture.detectChanges();
  });

  it('creates the component', () => {
    expect(component).toBeTruthy();
  });
});
```

8.2 User-facing DOM behavior

```
it('submits the entered name', async () => {
  const page: HTMLElement = fixture.nativeElement;
  const input = page.querySelector<HTMLInputElement>('#name')!;

  input.value = 'Ayesha';
  input.dispatchEvent(new Event('input', { bubbles: true }));
  page.querySelector<HTMLButtonElement>('button[type="submit"]')!.click();
  await fixture.whenStable();

  expect(page.textContent).toContain('Ayesha');
});
```

Prefer accessible selectors such as labels, roles and meaningful `aria-label` values. Verify rendered behavior as well as component state when the template is part of the requirement.

9. Services and HTTP mocks

9.1 Pure service

```
describe('PriceService', () => {
  let service: PriceService;

  beforeEach(() => {
    TestBed.configureTestingModule({});
    service = TestBed.inject(PriceService);
  });

  it('applies tax', () => {
    expect(service.withTax(100, 0.15)).toBe(115);
  });
});
```

9.2 HTTP service

```
beforeEach(() => {
  TestBed.configureTestingModule({
    providers: [
      UserService,
      provideHttpClient(),
      provideHttpClientTesting()
    ]
  });
  service = TestBed.inject(UserService);
  http = TestBed.inject(HttpTestingController);
});

afterEach(() => http.verify());

it('loads users', () => {
  service.getUsers().subscribe(users => {
    expect(users).toEqual([{ id: 1, name: 'Ayesha' }]);
  });

  const request = http.expectOne('/api/users');
  expect(request.request.method).toBe('GET');
  request.flush([{ id: 1, name: 'Ayesha' }]);
});

it('propagates server failure', () => {
  service.getUsers().subscribe({
    error: error => expect(error.status).toBe(500)
  });
  http.expectOne('/api/users').flush('Failed', {
    status: 500,
    statusText: 'Server Error'
  });
});
```

`HttpTestingController` prevents real network access and makes the response deterministic. `verify()` ensures no unexpected request remains.

10. Pipes, directives and utilities

10.1 Pure pipe

```
it('formats a display name', () => {
  const pipe = new DisplayNamePipe();
  expect(pipe.transform('ayesha', 'khan')).toBe('Ayesha Khan');
});
```

10.2 Directive through a host

```
@Component({
  imports: [HighlightDirective],
  template: '<p appHighlight>Text</p>'
})
class HostComponent {}

it('applies highlight styling', () => {
  const fixture = TestBed.createComponent(HostComponent);
  fixture.detectChanges();
  const element = fixture.nativeElement.querySelector('p');
  expect(element.style.backgroundColor).toBe('yellow');
});
```

10.3 Utility function

Test utilities directly without TestBed. Include normal, invalid and boundary values. Angular infrastructure adds no value to a pure function test.

11. Coverage policy

Metric	What it measures	Common gap
Statements	Executable instructions reached	Early-return validation not tested
Branches	Conditional alternatives reached	Only success path covered
Functions	Functions/handlers invoked	Unused callback or UI handler
Lines	Executable lines reached	Error handling line not reached

```
npm run test:coverage
```

Open the generated HTML report under `coverage/<project-name>/index.html`. Treat exclusions as reviewed policy. Do not exclude difficult business logic simply to raise the percentage. Coverage does not prove requirements are correct; assertions and scenario selection do.

12. Allure integration



12.1 Add report metadata

```
import * as allure from 'allure-js-commons';

beforeEach(async () => {
  await allure.epic('Customer portal');
  await allure.feature('User profile');
  await allure.severity('normal');
});
```

12.2 Generate and view

```
npm test
npm run allure:generate
npm run allure:open
```

`allure:generate` converts raw result files; it does not rerun tests. The viewer starts a local report server. Stop it with `Ctrl+C`. Avoid placing passwords, tokens or real customer data in attachments and parameters.

13. Quality gates

13.1 Result validation script

Create `scripts/check-test-results.mjs`. At minimum, read `test-results/vitest.json` and fail when there are zero tests, failures, skipped/todo results or an inconsistent count. Also compare Allure result count and require name, suite, status and timing metadata.

```
import { readFileSync } from 'node:fs';

const report = JSON.parse(
  readFileSync('test-results/vitest.json', 'utf8')
);
const tests = report.testResults.flatMap(
  suite => suite.assertionResults
);
const incomplete = tests.filter(test => test.status !== 'passed');

if (!report.success || tests.length === 0 || incomplete.length) {
  throw new Error(
    'Quality gate failed: every discovered test must pass; skips are not allowed.'
  );
}
```

13.2 Safe cleanup

Delete only fixed generated directories such as coverage, test-results, allure-results and allure-report. Never accept an arbitrary deletion path from a command argument. Reject symbolic links and paths outside the repository.

13.3 Gate sequence



Run the production build only after required checks succeed. A test report being visually green does not override a failed coverage or lint gate.

14. Linting and formatting

Apply Angular ESLint recommendations to TypeScript and templates. Add Vitest rules to spec files:

```
{
  files: ['**/*.spec.ts'],
  plugins: { vitest },
  languageOptions: {
    globals: vitest.environments.env.globals
  },
  rules: {
    ...vitest.configs.recommended.rules,
    'vitest/no-disabled-tests': 'error',
    'vitest/no-focused-tests': 'error'
  }
}
```

Ignore generated directories in ESLint, Prettier and Git:

```
node_modules/  
.angular/  
dist/  
coverage/  
test-results/  
allure-results/  
allure-report/
```

`npm run format:check` reports differences without editing files. `npm run format` modifies maintained source files.

15. GitHub Actions CI

Create `.github/workflows/quality.yml`. Store the team-approved Node version in `.nvmrc`, or replace `node-version-file` with a fixed version.

```
name: Quality checks

on:
  push:
  pull_request:
  workflow_dispatch:

permissions:
  contents: read

jobs:
  quality:
    runs-on: ubuntu-latest
    timeout-minutes: 15
    env:
      CI: 'true'
      NG_CLI_ANALYTICS: 'false'
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-node@v4
        with:
          node-version-file: '.nvmrc'
          cache: npm

      - name: Install locked dependencies
        run: npm ci

      - name: Lint
        run: npm run lint

      - name: Check formatting
        run: npm run format:check

      - name: Test, coverage and result gates
        id: tests
        run: npm run test:ci

      - name: Generate Allure report
        if: ${!cancelled()} && steps.tests.outcome != 'skipped'
        run: npm run allure:generate

      - name: Production build
        run: npm run build

      - name: Preserve reports
        if: ${always()}

```

```

uses: actions/upload-artifact@v4
with:
  name: test-reports-${{ github.run_id }}-${{ github.run_attempt }}
  path: |
    coverage/
    test-results/
    allure-results/
    allure-report/
  if-no-files-found: warn
  retention-days: 14

```

15.1 What each CI step proves

Step	Evidence
Checkout	The runner uses the committed repository state.
Setup Node	Local and CI runtime versions can be aligned.
npm ci	The lockfile installs successfully from scratch.
Lint/format	Static quality standards are met.
Test gate	Behavior, coverage and result-completeness rules pass.
Build	The validated source compiles for production.
Artifact upload	Evidence is retained for investigation and review.

15.2 Repository controls

After the workflow passes, configure branch protection so the *quality* job is required before merge. Limit workflow permissions and store deployment secrets in GitHub environments or repository secrets, never in source files.

16. Artifacts and CI diagnosis

1. Open the repository's **Actions** tab.
2. Select the workflow run associated with the commit.
3. Open the job and start with the first red step.
4. Read the assertion, lint, threshold or compiler error.
5. Download report artifacts for deeper inspection.
6. Reproduce the same npm command locally.
7. Fix the cause; do not disable the gate simply to make the run green.

An artifact SHA-256 value is an integrity fingerprint. It is not a website address. Uploaded reports are retained files; they are not automatically public Allure hosting.

17. Optional IIS deployment

This section applies when the Angular application can be deployed as static HTML, CSS and JavaScript on a Windows machine. SSR applications require a Node process or a supported reverse-proxy design.

17.1 Create a static build configuration

```
"configurations": {  
  "iis": {  
    "outputMode": "static",  
    "outputPath": "dist/<app-name>-iis"  
  }  
}
```

```
"build:iis": "ng build --configuration production,iis"
```

17.2 Enable IIS

Run from Administrator PowerShell:

```
Enable-WindowsOptionalFeature -Online `  
  -FeatureName IIS-WebServerRole,IIS-WebServer,IIS-CommonHttpFeatures,IIS-  
  StaticContent,IIS-DefaultDocument,IIS-ManagementConsole `  
  -All -NoRestart
```

17.3 Publish files

```
npm run build:iis  
  
New-Item -ItemType Directory `  
  -Path "C:\inetpub\<app-name>" -Force  
  
Copy-Item `  
  -Path ".\dist\<app-name>-iis\browser\*" `  
  -Destination "C:\inetpub\<app-name>" `  
  -Recurse -Force
```

17.4 Site configuration

- Create a dedicated application pool with *No Managed Code*.
- Create a site bound to an unused port, for example 127.0.0.1:8080.
- Enable Anonymous Authentication using the application-pool identity.
- Grant that identity Read and Execute permissions on the site folder.
- Ensure `index.html` is a default document.
- For client-side routes, add a reviewed URL Rewrite fallback to `index.html`.

```
icacls "C:\inetpub\<app-name>" `
  /grant "IIS AppPool\<pool-name>:(OI)(CI)(RX)" /T

curl.exe -i http://127.0.0.1:8080/
```

Scope: IIS on a developer workstation is a local deployment. It remains available when the IDE and Node terminal are closed, but only while Windows and IIS are running. It is not automatically public.

18. Verification checklist

Requirement	Manual verification	Expected result
Locked install	<code>npm ci</code>	Clean installation succeeds.
Tests	<code>npm test</code>	All discovered cases pass.
Watch mode	<code>npm run test:watch</code> , edit a spec	Affected tests rerun.
Coverage	<code>npm run test:coverage</code>	Reports created and thresholds pass.
No skipped tests	Temporarily add <code>it.skip</code>	Lint or result gate fails.
No focused tests	Temporarily add <code>it.only</code>	Vitest or lint fails.
Allure	Generate and open report	Suites, status and duration appear.
Quality pipeline	<code>npm run quality-check</code>	All gates and build pass in order.
CI	Push a branch or open a PR	GitHub quality job succeeds.
Artifacts	Download from workflow run	Coverage and Allure outputs exist.
IIS	Stop Node/IDE, refresh local URL	Static application remains available.

Probe-test rule: Run deliberate failure experiments only on a temporary branch or uncommitted change, then remove them and run the complete pipeline again.

19. Troubleshooting

Problem	Likely cause	Resolution
Coverage package missing	@vitest/coverage-v8 absent or incompatible	Install a version matching the Vitest major.
No tests found	Wrong file suffix/path or builder config	Use *.spec.ts; review include/exclude settings.
Test passes alone but fails in suite	Shared state, mock or timer leakage	Fresh setup; restore mocks/timers; remove ordering dependency.
DOM does not update	Angular work has not settled	Use detectChanges() or await whenStable().
Allure report is empty	Reporter missing or wrong results directory	Run tests first; verify reporter and paths.
Old results appear	Raw results were not cleaned	Clean generated directories before a final run.
Local passes, CI fails	Runtime, case sensitivity or lockfile difference	Match Node versions; use npm ci; inspect first failed step.
Windows EPERM during install	A process holds a dependency binary	Stop dev/watch sessions, then rerun installation.
IIS HTTP 401	Anonymous module/identity/ACL problem	Verify authentication modules and grant pool identity RX access.
IIS site state Unknown	WAS has stale runtime configuration	Restart WAS and W3SVC in the correct order.

20. Rollout and maintenance

20.1 Adoption sequence

1. Record the current Angular, Node and existing test setup.
2. Make the base Vitest suite pass before adding gates.
3. Add coverage and agree an honest threshold.
4. Add Allure reporting and deterministic cleanup.
5. Add lint/format rules and remove obsolete tests.
6. Add the complete local quality command.
7. Add GitHub Actions and verify both success and failure paths.
8. Make the CI job required in branch protection.
9. Add delivery only after the validation pipeline is stable.

20.2 Definition of done for every change

- New behavior has positive, negative and relevant edge cases.
- External dependencies are mocked at the boundary.
- No test depends on execution order or real production data.

- Lint, format, tests, coverage and build pass locally.
- CI passes on the pushed commit.
- Reports contain no skipped, stale or missing results.
- Documentation and commands remain accurate.

20.3 Review principles

Prefer tests that express business behavior over tests that mirror implementation. Review exclusions and thresholds periodically. Upgrade dependencies in controlled pull requests. Re-run the failure probes after major runner, Angular or reporting upgrades.

21. Copy-paste support scripts

21.1 Safe report cleanup

Create scripts/clean-reports.mjs:

```
import { lstatSync, mkdirSync, realpathSync, rmSync } from 'node:fs';
import { dirname, isAbsolute, join, relative } from 'node:path';
import { fileURLToPath } from 'node:url';

const root = realpathSync(join(dirname(fileURLToPath(import.meta.url)), '..'));
const groups = {
  results: ['allure-results', 'test-results', 'coverage'],
  report: ['allure-report'],
  all: ['allure-results', 'allure-report', 'test-results', 'coverage']
};

const group = process.argv[2] ?? 'all';
if (!Object.hasOwn(groups, group)) {
  throw new Error('Use results, report, or all.');
```

```
}

for (const name of groups[group]) {
  const target = join(root, name);
  const withinRoot = relative(root, target);

  if (!withinRoot || withinRoot.startsWith '..') || isAbsolute(withinRoot)) {
    throw new Error(`Refusing to clean outside the project: ${target}`);
  }

  const stat = lstatSync(target, { throwIfNoEntry: false });
  if (stat?.isSymbolicLink()) {
    throw new Error(`Refusing to clean a linked directory: ${target}`);
  }

  if (stat && realpathSync(target) !== target) {
    throw new Error(`Unexpected resolved path: ${target}`);
  }

  rmSync(target, { recursive: true, force: true });
  console.log(`Cleaned ${name}`);
}

mkdirSync(join(root, 'test-results'), { recursive: true });
```

Add cleanup hooks to package.json:

```
"pretest": "node scripts/clean-reports.mjs results",  
"pretest:watch": "node scripts/clean-reports.mjs results",  
"pretest:coverage": "node scripts/clean-reports.mjs results",  
"preallure:generate": "node scripts/clean-reports.mjs report"
```

21.2 Test and Allure completeness gate

Create scripts/check-test-results.mjs:

```

import { readFileSync, readdirSync } from 'node:fs';
import { fileURLToPath } from 'node:url';

const root = new URL('../', import.meta.url);
const report = JSON.parse(
  readFileSync(new URL('test-results/vitest.json', root), 'utf8')
);
const tests = report.testResults.flatMap(suite => suite.assertionResults);
const incomplete = tests.filter(test => test.status !== 'passed');

if (
  !report.success ||
  report.numTotalTests === 0 ||
  tests.length !== report.numTotalTests ||
  report.numPassedTests !== report.numTotalTests ||
  incomplete.length > 0
) {
  for (const test of incomplete) {
    console.error(`${test.status}: ${test.fullName}`);
  }
  throw new Error(
    'Every discovered test must pass. Skipped/todo tests are not allowed.'
  );
}

const resultsDirectory = new URL('allure-results/', root);
const results = readdirSync(resultsDirectory)
  .filter(name => name.endsWith('-result.json'))
  .map(name =>
    JSON.parse(readFileSync(new URL(name, resultsDirectory), 'utf8'))
  );

const invalidAllureResult = results.some(result =>
  result.status !== 'passed' ||
  !Number.isFinite(result.start) ||
  !Number.isFinite(result.stop) ||
  !result.fullName ||
  !result.labels?.some(label =>
    ['parentSuite', 'suite', 'subSuite'].includes(label.name)
  )
);

if (results.length !== tests.length || invalidAllureResult) {
  throw new Error(
    'Allure must contain a passing, timed, named result with a suite for every test.'
  );
}

```

```
console.log(  
  `Quality gate passed: ${tests.length} tests passed, none skipped.`  
);  
console.log(`Allure results: ${fileURLToPath(resultsDirectory)}`);
```

21.3 Full ESLint outline

Adapt package imports to the versions selected by your project:

```

import eslint from '@eslint/js';
import tseslint from 'typescript-eslint';
import angular from 'angular-eslint';
import vitest from '@vitest/eslint-plugin';
import globals from 'globals';

export default tseslint.config(
  {
    ignores: [
      'node_modules/**', '.angular/**', 'dist/**', 'coverage/**',
      'allure-results/**', 'allure-report/**', 'test-results/**'
    ]
  },
  {
    files: ['**/*.ts'],
    extends: [
      eslint.configs.recommended,
      ...tseslint.configs.recommended,
      ...angular.configs.tsRecommended
    ],
    processor: angular.processInlineTemplates,
    languageOptions: { globals: { ...globals.node, ...globals.browser } }
  },
  {
    files: ['src/**/*.html'],
    extends: [
      ...angular.configs.templateRecommended,
      ...angular.configs.templateAccessibility
    ]
  },
  {
    files: ['**/*.spec.ts'],
    plugins: { vitest },
    languageOptions: { globals: vitest.environments.env.globals },
    rules: {
      ...vitest.configs.recommended.rules,
      'vitest/no-disabled-tests': 'error',
      'vitest/no-focused-tests': 'error'
    }
  }
);

```

Template notice: These files are implementation starting points. Run them in the target repository, review paths and generated JSON shapes, and keep dependency versions locked. Major upgrades can change configuration APIs or reporter output.

22. Glossary and references

Term	Meaning
Assertion	A statement of the expected result.
Mock	A controlled replacement for a dependency.
Fixture	Angular wrapper around a component instance and rendered DOM.
Coverage	Measurement of source executed by tests.
Quality gate	A rule that returns failure when a standard is not met.
Reporter	A component that formats or stores test results.
Artifact	A file bundle retained from a CI run.
Runner	The machine that executes a CI job.
CI	Automated validation of integrated changes.
CD	Automated delivery or deployment of validated software.

Primary references

- [Angular unit testing](#)
- [Angular code coverage](#)
- [Angular Karma-to-Vitest migration](#)
- [Vitest configuration](#)
- [Allure Vitest integration](#)
- [GitHub Actions for Node.js](#)
- [Microsoft IIS static website guide](#)